

UDP (User Datagram Protocol)

UDP (User Datagram Protocol) হচ্ছে TCP/IP প্রোটোকল স্যুটের একটি ট্রান্সপোর্ট লেয়ার প্রোটোকল।

✦ কোথায় বসে আছে UDP?

- UDP অ্যাপ্লিকেশন লেয়ারের নিচে এবং IP লেয়ারের উপরে থাকে।
- মানে, অ্যাপ্লিকেশন প্রোগ্রাম যেসব ডেটা পাঠায়, সেগুলো নেটওয়ার্কে পাঠানোর আগে UDP সামলায়, আবার নেটওয়ার্ক থেকে ডেটা এলে অ্যাপ্লিকেশনকে পৌঁছে দেয়।

UDP এর কাজ

1. Process-to-Process Communication

- TCP/UDP এর মূল কাজ হচ্ছে "প্রসেস-টু-প্রসেস" যোগাযোগ।
- এর জন্য UDP port number ব্যবহার করে।

2. Control Mechanism

- TCP এর মতো শক্তিশালী কন্ট্রোল মেকানিজম UDP তে নেই।
- UDP তে flow control নেই (মানে বেশি ডেটা পাঠানো হলে কনজেশন কন্ট্রোল করে না)।
- acknowledgment নেই (প্যাকেট পৌঁছেছে কিনা সে খবর দেয় না)।
- তবে error check করে, যদি কোনো প্যাকেটে সমস্যা থাকে, সেটা ড্রপ করে দেয়, কিন্তু আবার পাঠায় না।

UDP এর বৈশিষ্ট্য

- UDP হলো connectionless → আগে থেকে কোনো কানেকশন বানায় না, সরাসরি প্যাকেট পাঠিয়ে দেয়।
- UDP হলো unreliable → প্যাকেট ঠিকমতো পৌঁছাবে কিনা, সেটা গ্যারান্টি দেয় না।
- UDP এর কাজ খুবই সিম্পল, TCP এর মতো জটিল প্রক্রিয়া নেই।

UDP কেন ব্যবহার করা হয়?

অনেকে প্রশ্ন করে — যদি UDP এতটাই দুর্বল হয়, তাহলে ব্যবহার করার দরকার কী?

👉 কারণ এর কিছু সুবিধাও আছে:

- খুব কম overhead লাগে।
- ছোট ডেটা/মেসেজ পাঠাতে UDP ভালো।
- TCP এর মতো বেশি ধাপ নেই, তাই UDP দিয়ে ডেটা পাঠানো অনেক দ্রুত এবং সহজ।

সংক্ষেপে

UDP ব্যবহার করা হয় যখন —

- ডেটা ছোট,
- দ্রুত পাঠাতে হবে,
- reliability খুব গুরুত্বপূর্ণ না।

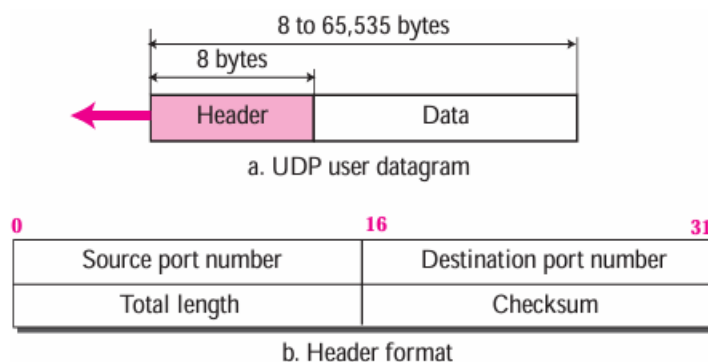
✦ উদাহরণ: DNS query, voice/video streaming, online games — এগুলোতে UDP বেশি ব্যবহৃত হয়।

UDP User Datagram

UDP-তে যেসব প্যাকেট পাঠানো হয়, সেগুলোকে বলা হয় **User Datagram**।

- প্রতিটি ইউজার ড্যাটাগ্রামের header size ফিক্সড 8 bytes।
- হেডারের পরেই থাকে আসল ডেটা (payload)।

Figure 14.2 User datagram format



Header এর ৪টি ফিল্ড

1 Source Port Number (16 bits)

- যেই কম্পিউটার/ডিভাইস ডেটা পাঠাচ্ছে, তার **process** কোন পোর্ট ব্যবহার করছে সেটা বোঝায়।
 - Port range: 0 - 65535
 - যদি **client** হয় → সাধারণত **ephemeral port** (অস্থায়ী, random ভাবে বেছে নেয়)।
 - যদি **server** হয় → সাধারণত **well-known port** (যেমন DNS = 53, HTTP = 80)।
-

2 Destination Port Number (16 bits)

- ডেটা যে ডিভাইস/প্রসেসে যাবে, সেটার পোর্ট নাম্বার।
 - Port range: 0 - 65535
 - যদি **server** গন্তব্য হয় (client → server request) → সাধারণত **well-known port** ব্যবহার হবে।
 - যদি **client** গন্তব্য হয় (server → client response) → সাধারণত **ephemeral port** ব্যবহার হবে (server, client এর দেওয়া port number কপি করে পাঠায়)।
-

3 Length (16 bits)

- পুরো **UDP ডাটাগ্রামের (Header + Data) দৈর্ঘ্য কত bytes, সেটা দেখায়।**
 - সর্বোচ্চ দৈর্ঘ্য = 65,535 bytes (কিন্তু বাস্তবে তার চেয়ে কম হতে হয় কারণ এটা IP ডাটাগ্রামের ভেতরে থাকে)।
 - আসলে **এই ফিল্ডটা অতিরিক্ত (redundant), কারণ IP ডাটাগ্রামের ভেতরেও total length আর header length থাকে।**
 - অর্থাৎ, **UDP length = IP length - IP header length**
 - তবুও UDP-তে আলাদা length ফিল্ড রাখা হয়েছে efficiency বাড়ানোর জন্য (IP layer এর কাছে না গিয়ে সরাসরি UDP layer হিসাব করতে পারে)।
-

4 Checksum (16 bits)

- পুরো ডাটাগ্রাম (header + data) এর **error detection** এর জন্য ব্যবহার হয়।
 - যদি কোনো বিট error পাওয়া যায় → UDP সেই প্যাকেট **drop করে দেয়।**
 - Error থাকলে retransmission হয় না (কারণ UDP unreliable)।
-

ঠিক আছে 😊 Example 14.1 টা বাংলায় ধাপে ধাপে বুঝিয়ে দিচ্ছি:

আমাদের দেওয়া UDP header dump (hexadecimal):

CB84 000D 001C 001C

Step by Step Analysis

a. Source Port Number

- প্রথম ৪ হেক্স ডিজিট = CB84
 - Hex $\rightarrow$ Decimal: $CB84_{16} = 52100_{10}$
☒ Source Port = **52100**
-

b. Destination Port Number

- পরের ৪ হেক্স ডিজিট = 000D
 - Hex $\rightarrow$ Decimal: $000D_{16} = 13_{10}$
☒ Destination Port = **13**
-

c. Total Length (UDP packet)

- তৃতীয় ৪ হেক্স ডিজিট = 001C
 - Hex $\rightarrow$ Decimal: $001C_{16} = 28_{10}$
☒ Total Length = **28 bytes**
-

d. Data Length

- UDP Header সবসময় 8 bytes
 - Data length = Total length – Header length
 $= 28 - 8 = 20 \text{ bytes}$
☒ Data Length = **20 bytes**
-

e. Client or Server?

- Destination port number = **13**
- Port 13 হলো **well-known port** (Daytime service) $\rightarrow$ সাধারণত সার্ভার ব্যবহার করে।

- মানে এটা Client → Server packet।
☒ Packet যাচ্ছে Client থেকে Server এ

f. Client Process

- Port 13 এর সার্ভিস হলো **Daytime Protocol** (RFC 867 অনুযায়ী)।
☒ Client Process = **Daytime service**

Final Answer (বাংলায়)

- a) Source port number = **52100**
- b) Destination port number = **13**
- c) UDP packet এর মোট দৈর্ঘ্য = **28 bytes**
- d) ডেটার দৈর্ঘ্য = **20 bytes**
- e) প্যাকেট যাচ্ছে **Client → Server**
- f) Client process = **Daytime service**

UDP এর Services

1) Process-to-Process Communication

- **UDP socket** ব্যবহার করে প্রসেস-টু-প্রসেস যোগাযোগ করে।
- **Socket = IP Address + Port Number** এর কম্বিনেশন।
- প্রতিটি অ্যাপ্লিকেশন/প্রসেস আলাদা পোর্ট নাম্বার ব্যবহার করে।

✦ UDP এর কিছু **well-known ports** (Table 14.1 থেকে):

- **7 → Echo** : যা প্যাকেট পায়, সেটাকে আবার sender এ ফেরত পাঠায়।
- **9 → Discard** : যা প্যাকেট পায়, সেটাকে ফেলে দেয়।
- **13 → Daytime** : তারিখ ও সময় ফেরত দেয়।
- **17 → Quote** : দিনের কোট/বাণী ফেরত দেয়।
- **19 → Chargen** : অক্ষরের স্ট্রিং জেনারেট করে ফেরত দেয়।
- **53 → DNS** : Domain Name Service।
- **67, 68 → BOOTP** : বুটস্ট্র্যাপ ইনফো ডাউনলোডের জন্য সার্ভার/ক্লায়েন্ট।
- **69 → TFTP** : Trivial File Transfer Protocol।
- **123 → NTP** : Network Time Protocol।

- 161, 162 → SNMP : Simple Network Management Protocol।
-

2 Connectionless Services

- UDP হলো connectionless protocol।
 - প্রতিটি user datagram আলাদা এবং স্বাধীন।
 - কোনো sequence number থাকে না।
 - connection establish বা terminate করার দরকার হয় না (TCP এর মতো না)।
 - এর মানে:
 - একই source → একই destination এ গেলেও প্যাকেটগুলো একে অপরের সাথে সম্পর্কিত নয়।
 - প্রতিটি datagram নেটওয়ার্কে আলাদা path নিতে পারে।
 - UDP-তে বড় ডেটা স্ট্রিম ভাগ করে পাঠানো যায় না।
 - তাই UDP কেবল ছোট মেসেজ ($\leq 65,507$ bytes) পাঠানোর জন্য ব্যবহার করা হয়।
-

3 Flow Control

- UDP তে flow control নেই।
 - মানে:
 - যদি sender অনেক বেশি ডেটা পাঠায়, receiver overflow হয়ে যেতে পারে।
 - TCP এর মতো window mechanism UDP তে নেই।
 - তাই flow control লাগলে application/protocol নিজে implement করতে হবে।
-

4 Error Control

- UDP তে কেবলমাত্র checksum দিয়ে error detection হয়।
 - যদি ডেটা corrupt হয়, receiver চুপচাপ discard করে দেয়।
 - Retransmission নেই, acknowledgment নেই।
 - মানে sender জানতেই পারবে না যে ডেটা পৌঁছেছে নাকি হারিয়ে গেছে।
 - তাই যেসব অ্যাপ্লিকেশন error recovery চায়, তাদেরকে নিজে থেকে এটা handle করতে হবে।
-

সংক্ষেপে

- UDP Services = খুবই simple

1. **Process-to-process communication** (port number দিয়ে)
2. **Connectionless transmission** (প্রতিটি packet আলাদা)
3. **No flow control** → receiver overflow হতে পারে
4. **No error control (except checksum)** → error হলে প্যাকেট ড্রপ হয়ে যায়

👉 তাই UDP কেবল সেই সব কাজের জন্য ভালো, যেখানে speed গুরুত্বপূর্ণ আর reliability খুব বেশি দরকার নেই (যেমন: DNS, voice/video streaming, gaming)।

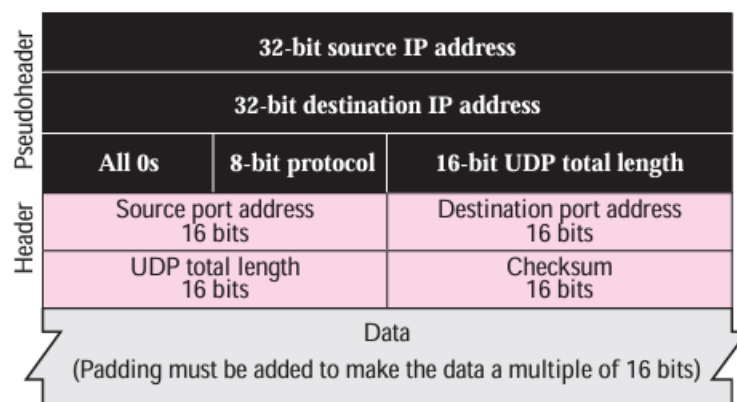
চমৎকার প্রশ্ন 👍 এবার আমি তোমাকে UDP Checksum বাংলায় সহজভাবে ব্যাখ্যা করছি।

UDP Checksum কীভাবে কাজ করে?

UDP checksum হচ্ছে **error detection** মেকানিজম, যা UDP প্যাকেট (header + data) এর সাথে আরও কিছু extra তথ্য যোগ করে হিসাব করা হয়।

- ♦ UDP checksum গণনার সময় ৩টি জিনিস যোগ হয়:

Figure 14.3 Pseudoheader for checksum calculation



1. Pseudoheader

- IP header-এর কিছু অংশ নিয়ে বানানো হয়।
- এতে থাকে:

- Source IP address (32-bit)
- Destination IP address (32-bit)
- Reserved (সব 0) (8-bit)
- Protocol (UDP = 17) (8-bit)
- UDP length (16-bit)

👉 এর উদ্দেশ্য হলো এটা ensure করা যে প্যাকেটটা সঠিক source → সঠিক destination → সঠিক protocol (UDP) এ পৌঁছাচ্ছে।

2. UDP Header

- Source Port (16-bit)
- Destination Port (16-bit)
- UDP Length (16-bit)
- UDP Checksum (16-bit) → এটা হিসাব করার সময় initially 0 ধরা হয়।

3. UDP Data (Payload)

- Application layer থেকে আসা ডেটা।
- যদি ডেটার দৈর্ঘ্য odd হয় (যেমন 7 byte), তখন padding (একটি 0 byte) যোগ করা হয় যাতে এটি 16-bit multiple হয়ে যায়।

Checksum কাজ করার নিয়ম

1. Sender → pseudoheader + UDP header + data একসাথে যোগ করে 16-bit words আকারে।
2. সব 16-bit word কে one's complement addition দিয়ে যোগ করা হয়।
3. Final sum এর one's complement নেওয়া হয় = Checksum value।
4. Receiver → আবার একইভাবে checksum গণনা করে।
 - যদি ফলাফল 1111 1111 1111 1111 (সব 1) হয় → data error-free।
 - যদি অন্য কিছু হয় → error আছে → UDP সেই প্যাকেট discard করে দেয়।

কেন Pseudoheader দরকার?

- ধরো, UDP datagram ঠিকঠাক এসেছে। কিন্তু যদি IP header-এ destination IP ভুল হয়ে যায়, তাহলে ডেটা ভুল host এ পৌঁছাতে পারে।
- তাই pseudoheader-এর মধ্যে source IP, destination IP, protocol নম্বর দেওয়া থাকে → এগুলিও checksum-এ যোগ হয়।
- যদি এর মধ্যে কোনোটা corrupt হয়, checksum মিসম্যাচ হবে → packet drop হবে।

Protocol Field এর মান

- UDP এর protocol নম্বর = 17।
- যদি transmission চলাকালে protocol number পরিবর্তিত হয় → checksum detect করবে এবং packet drop করবে।

Optional Checksum

- UDP-তে checksum **optional**।
- Sender চাইলে checksum না-ও করতে পারে → তখন checksum field = 0 রাখা হয়।
- যদি checksum calculate করার পর result 0 হয় → সেটাকে all 1s এ (FFFF hex) পরিবর্তন করে পাঠানো হয়।
- কারণ naturally checksum কখনোই সব 1s হয় না, তাই এটা special case হিসেবে ব্যবহার করা হয়।

Example 14.2 (সংক্ষেপে)

- যদি ডেটা = 7 byte হয় → padding দিয়ে 8 byte করা হবে।
- Pseudoheader + UDP header + Data + Padding নিয়ে checksum করা হবে।
- Checksum এবং pseudoheader প্যাকেটের সাথে নেটওয়ার্কে পাঠানো হয় না, শুধু error-check করার জন্য ব্যবহৃত হয়।

সংক্ষেপে

- ☒ UDP checksum = **Pseudoheader + UDP Header + Data**
 - ☒ এর উদ্দেশ্য = **Error detection + Wrong host/protocol detection**
 - ☒ Optional (sender চাইলে বাদ দিতে পারে, তখন field = 0)
-

Figure 14.4 Checksum calculation of a simple UDP user datagram

153.18.8.105			
171.2.14.10			
All 0s	17	15	
1087		13	
15		All 0s	
T	E	S	T
I	N	G	Pad

10011001	00010010	→	153.18
00001000	01101001	→	8.105
10101011	00000010	→	171.2
00001110	00001010	→	14.10
00000000	00010001	→	0 and 17
00000000	00001111	→	15
00000100	00111111	→	1087
00000000	00001101	→	13
00000000	00001111	→	15
00000000	00000000	→	0 (checksum)
01010100	01000101	→	T and E
01010011	01010100	→	S and T
01001001	01001110	→	I and N
01000111	00000000	→	G and 0 (padding)
10010110 11101011		→	Sum
01101001 00010100		→	Checksum

ঠিক আছে ভাই ✨ — এবার আমি Figure 14.4 এর **UDP checksum calculation** টা ধাপে ধাপে full যোগফল দেখাই।

1. যেসব অংশ যোগ হবে (Checksum calculation এর জন্য)

UDP checksum এ তিনটা অংশ যোগ হয়:

1. **Pseudoheader** (Source IP, Destination IP, Protocol, UDP length)
2. **UDP Header** (Source port, Destination port, Length, Checksum=0)
3. **Data** (Application layer এর ডাটা, যদি odd হয় → padding)

2. Binary (16-bit words) গুলো

Figure এ already দেওয়া আছে, আমি grouping করে লিখলাম:

(a) Source IP (153.18.8.105) → 4 bytes → 2 words

10011001 00010010 → 153.18

00001000 01101001 → 8.105

(b) Destination IP (171.2.14.10) → 2 words

10101011 00000010 → 171.2

00001110 00001010 → 14.10

(c) Protocol + Zero + UDP Length (17, 15)

00000000 00010001 → Protocol = 17

00000000 00001111 → UDP Length = 15

(d) UDP Header (Source Port=1087, Dest Port=13, Length=15, Checksum=0)

```
00000100 00111111 → 1087
00000000 00001101 → 13
00000000 00001111 → Length = 15
00000000 00000000 → Checksum (initially 0)
```

(e) Data = "TESTING" (7 bytes → need 1 padding byte)

ASCII values → pair করে:

```
01010100 01000101 → T E
01010011 01010100 → S T
01001001 01001110 → I N
01000111 00000000 → G 0 (padding)
```

3. এখন সবগুলো যোগ করি (One's Complement Addition)

(সব 16-bit word যোগ করতে হবে)

👉 Step by step:

```
10011001 00010010 (153.18)
+00001000 01101001 (8.105)
=10100001 01111011
10100001 01111011
+10101011 00000010 (171.2)
=01001100 10011101 (carry wrap-around)
01001100 10011101
+00001110 00001010 (14.10)
=01011010 10100111
01011010 10100111
+00000000 00010001 (protocol=17)
=01011010 10111000
01011010 10111000
+00000000 00001111 (length=15)
=01011010 11000111
01011010 11000111
+00000100 00111111 (source port=1087)
=01011111 00000110
01011111 00000110
+00000000 00001101 (dest port=13)
=01011111 00010011
01011111 00010011
+00000000 00001111 (length=15)
=01011111 00100010
01011111 00100010
+00000000 00000000 (checksum=0)
=01011111 00100010
```

4. Data যোগ করি

```

01011111 00100010
+01010100 01000101 (T E)
=10110011 01100111
 10110011 01100111
+01010011 01010100 (S T)
=00000110 10111011 (carry wrap-around)
 00000110 10111011
+01001001 01001110 (I N)
=01010000 00001001
 01010000 00001001
+01000111 00000000 (G 0 padding)
=10010111 00001001

```

5. Final Sum

Sum = 10010111 00001001

6. One's Complement → Final Checksum

Checksum = 01101000 11110110

☒ তাই UDP checksum = **68F6 (hex)**

◇ Congestion Control (জ্যাম নিয়ন্ত্রণ)

- UDP যেহেতু connectionless protocol, তাই এতে **কোনও congestion control নেই**।
 - TCP এর মতো window, acknowledgment, বা rate control কিছুই UDP ব্যবহার করে না।
 - UDP ধরে নেয় যে packet গুলো ছোট ও কম সময়ে পাঠানো হয়, তাই নেটওয়ার্কে জ্যাম সৃষ্টি করবে না।
 - কিন্তু আজকাল **real-time audio ও video streaming** এ UDP ব্যবহার হয় → সেখানে অনেক বেশি packet একসাথে যায়, ফলে congestion হওয়ার সম্ভাবনা থাকে।
-

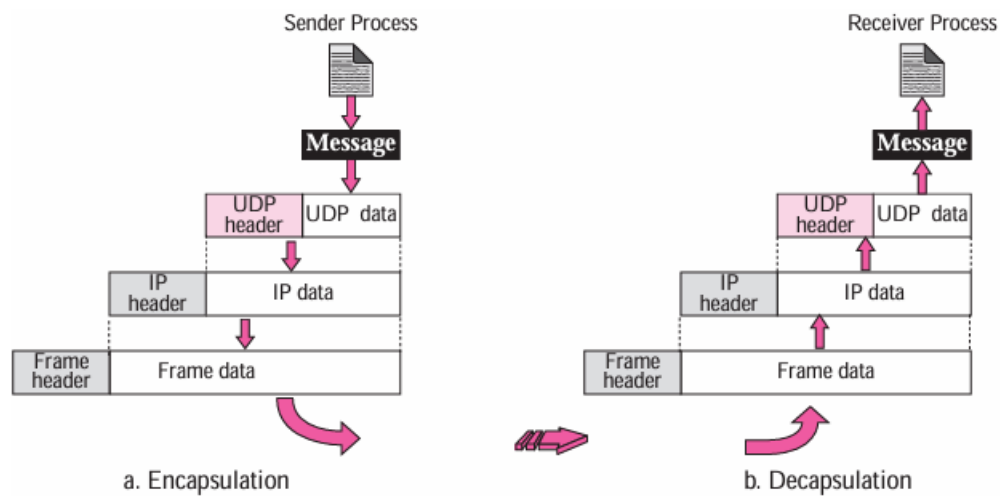
◇ Encapsulation (প্যাকেট মোড়ানো)

একটা message sender থেকে receiver এ পাঠানোর সময় প্রতিটি layer তার নিজের header (এবং কখনও trailer) যোগ করে → এটাকে বলা হয় encapsulation।

1. **Application Layer** → data তৈরি করে।

2. **UDP Layer** → application data এর সাথে UDP header যোগ করে → তৈরি হয় **UDP datagram**।
3. **IP Layer** → UDP datagram এর সাথে IP header যোগ করে → protocol field এ 17 বসায় (মানে data এসেছে UDP থেকে)।
4. **Data Link Layer** → IP datagram এর সাথে তার own header (এবং trailer) যোগ করে → তৈরি হয় **frame**।
5. **Physical Layer** → frame কে bits → electrical/optical signal এ রূপান্তর করে পাঠিয়ে দেয়।

Figure 14.5 Encapsulation and decapsulation



◇ Decapsulation (প্যাকেট খোলা)

Receiver এর দিক থেকে process উল্টা পথে চলে। প্রতিটি layer header/trailer খুলে data উপরে পাঠায়।

1. **Physical Layer** → signal কে bits এ decode করে data link layer এ দেয়।
2. **Data Link Layer** → তার own header/trailer check করে error আছে কিনা, error না থাকলে strip করে IP datagram পাঠায়।
3. **IP Layer** → IP header check করে (যেমন destination address ঠিক আছে কিনা), header ফেলে UDP datagram পাঠায়।
4. **UDP Layer** → checksum verify করে, header ফেলে application data দেয়। সঙ্গে sender এর socket address দেয় (যাতে reply করা যায়)।
5. **Application Layer** → data process করে।

✦ সহজভাবে বলতে গেলে →

Encapsulation = মোড়ানো (header যোগ করা)

Decapsulation = খোলা (header ফেলে data পাওয়া)

ভালো প্রশ্ন 👍

তুমি জানতে চাইছো — Queuing জিনিসটা আসলে কী?

◊ সাধারণভাবে Queue

Queue মানে হলো লাইন বা অপেক্ষমাণ তালিকা।
যেমন ধরো —

- ব্যাংকে টাকা তুলতে গিয়ে লাইন ধরতে হয়।
- আগে যিনি লাইনে দাঁড়ান, তিনি আগে সার্ভিস পান।

এটাই হলো FIFO (First In, First Out) প্রক্রিয়া → যে আগে আসবে, সে আগে যাবে।

◊ কম্পিউটার নেটওয়ার্কে Queue

Networking-এ Queue হলো একটি buffer (temporary storage) যেখানে ডেটা (packet বা message) কিছু সময়ের জন্য রাখা হয়, যতক্ষণ না সেটা প্রসেস বা পাঠানো হচ্ছে।

UDP-তে Queue এর কাজ:

1. Incoming Queue →

- বাইরের থেকে যে মেসেজ (datagram) আসে, সেগুলো প্রথমে এই queue তে রাখা হয়।
- Process (client/server program) যখন ready হয়, তখন সেখান থেকে একে একে মেসেজ নেয়।

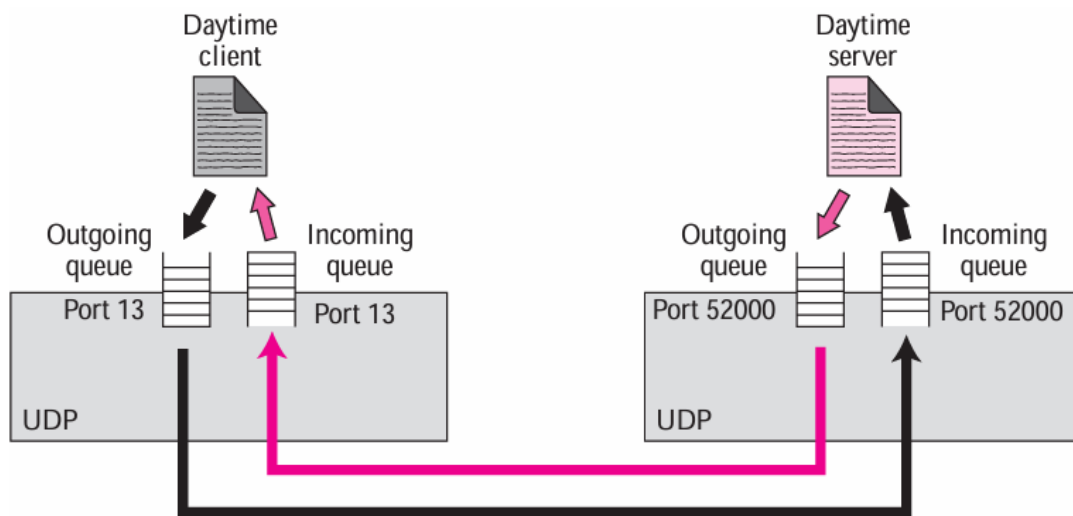
2. Outgoing Queue →

- Process যখন মেসেজ পাঠাতে চায়, সেটি outgoing queue তে জমা হয়।
- UDP/IP একে একে queue থেকে মেসেজ নিয়ে পাঠায়।

◇ কেন Queue দরকার?

- একসাথে অনেক মেসেজ আসতে পারে বা পাঠাতে হতে পারে।
- CPU বা নেটওয়ার্ক সব একসাথে handle করতে পারে না।
- তাই incoming/outgoing data কে সাময়িকভাবে line এ দাঁড় করিয়ে রাখা হয় (queue করা হয়)।
- এতে ডেটা হারিয়ে না গিয়ে সঠিক ক্রমে (FIFO) প্রসেস হয়।

Figure 14.6 Queues in UDP



◇ UDP Queue Concept

UDP-তে প্রতিটি port এর সাথে queue যুক্ত থাকে।

এই queue গুলো incoming (পাওয়া মেসেজ রাখার জন্য) এবং outgoing (পাঠানোর মেসেজ রাখার জন্য) দুই ধরনের হতে পারে।

◇ Client Side Queue

1. Port Allocation →

- যখন client-side কোনো process শুরু হয়, তখন OS তাকে একটি ephemeral port number দেয়।

- এই port এর সাথে queue তৈরি হয়।
 - 2. **Outgoing Queue** →
 - Client process মেসেজ পাঠায় → outgoing queue তে জমা হয়।
 - UDP header যোগ করে UDP সেই মেসেজ IP-তে পাঠায়।
 - ⚠️ যদি outgoing queue ভরে যায় → OS client process কে অপেক্ষা করতে বলে (block করে)।
 - 3. **Incoming Queue** →
 - Server থেকে আসা datagram UDP দেখে → যদি সেই port এর incoming queue থাকে → মেসেজ queue তে ঢোকে।
 - যদি queue না থাকে → UDP মেসেজ drop করে এবং **ICMP port unreachable** পাঠায়।
 - সব incoming মেসেজ (যেই server থেকেই আসুক) একই queue তে জমা হয়।
 - ⚠️ যদি incoming queue overflow হয় → UDP datagram drop করে + ICMP port unreachable পাঠায়।
-

◇ Server Side Queue

- 1. **Well-Known Port** →
 - Server শুরু হওয়ার সময় তার **well-known port** এর জন্য incoming ও outgoing queue তৈরি করে।
 - Queue সার্ভারের পুরো session ধরে খোলা থাকে।
 - 2. **Incoming Queue** →
 - Client থেকে আসা datagram যদি server এর port এর সাথে মিলে যায় → incoming queue তে জমা হয়।
 - যদি port না মেলে → UDP মেসেজ drop করে + ICMP port unreachable পাঠায়।
 - ⚠️ Incoming queue overflow → মেসেজ drop + ICMP port unreachable পাঠানো হয়।
 - 3. **Outgoing Queue** →
 - Server যখন reply দেয়, মেসেজ outgoing queue তে জমা হয়।
 - UDP header যোগ করে datagram IP তে পাঠানো হয়।
 - ⚠️ Outgoing queue overflow → OS server কে অপেক্ষা করতে বলে।
-

◇ সহজভাবে Flow

- **Client** ephemeral port ব্যবহার করে, **Server** well-known port ব্যবহার করে।
- প্রতিটি port এর সাথে queue থাকে (incoming + outgoing)।
- Queue overflow হলে → মেসেজ drop হয় বা process কে wait করতে হয়।
- Queue না থাকলে → UDP datagram drop হয় এবং **ICMP port unreachable** পাঠানো হয়।

চল, এবার **Multiplexing** আর **Demultiplexing** UDP-র ক্ষেত্রে বুঝি 🙌

◇ Multiplexing (Sender Side)

- একটি **host** এ অনেকগুলো process (program) একসাথে UDP ব্যবহার করতে পারে (যেমন: DNS, NTP, TFTP ইত্যাদি)।
- কিন্তু আসলে UDP software **একটাই** থাকে।
- **তাই অনেকগুলো process → এক UDP এই many-to-one সম্পর্ককে বলে Multiplexing।**
- UDP প্রতিটি process কে **port number** দিয়ে আলাদা করে।
- প্রতিটি process → মেসেজ পাঠায় → UDP header (source port + destination port) যোগ করে → IP layer এ পাঠায়।

🙌 একে বলা যায়:

"অনেকগুলো sender process এর মেসেজ UDP নিয়ে একটা লাইন (queue) এ সাজিয়ে IP তে পাঠায়।"

◇ Demultiplexing (Receiver Side)

- Receiver host এও UDP software **একটাই** থাকে।
- কিন্তু সে host এ একাধিক process থাকতে পারে যারা UDP দিয়ে মেসেজ পেতে চায়।
- তাই **এক UDP → অনেক process** এই one-to-many সম্পর্ককে বলে **Demultiplexing।**
- UDP যখন IP থেকে মেসেজ পায় →
 1. Header এর **destination port number** দেখে
 2. সেই অনুযায়ী সঠিক process এর কাছে মেসেজ পাঠিয়ে দেয়।

🙌 একে বলা যায়:

"UDP মেসেজ গুলো আলাদা আলাদা process এর হাতে পৌঁছে দেয়, port number দেখে।"

◇ UDP vs Simple Protocol

- UDP আসলে Chapter 13 এর **connectionless simple protocol** এর মতো।
- পার্থক্য হলো UDP তে **optional checksum** আছে → ডেটা corruption হলে সেটা detect করে discard করে।

- তবে error হলে কোনো **feedback (acknowledgment)** পাঠায় না।
-

👉 এক কথায়:

- **Multiplexing** = অনেক sender process → এক UDP
 - **Demultiplexing** = এক UDP → অনেক receiver process
-

ঠিক আছে ভাই 😊—UDP (User Datagram Protocol) নিয়ে তুমি যে অংশ পাঠিয়েছো সেটাকে আমি একদম সহজ করে, উদাহরণসহ, ধাপে ধাপে বুঝিয়ে দিচ্ছি।

◇ UDP কী?

UDP হলো একটা **transport layer protocol** (TCP/IP এর ৪ নম্বার লেয়ারের একটা প্রোটোকল)। এটা **connectionless** – মানে আগে থেকে কোন connection তৈরি করতে হয় না। যেটা পাঠাতে চাই সরাসরি পাঠিয়ে দিই।

তবে downside হলো এটা **unreliable**—মানে error checking বা ডেলিভারি নিশ্চিত করার মতো সিস্টেম UDP তে নেই (শুধু optional checksum আছে)।

◇ Multiplexing & Demultiplexing (UDP এর কাজ)

একটা host-এ অনেকগুলো **process (software/application)** UDP ব্যবহার করতে চায়। কিন্তু সিস্টেমে UDP একটাই থাকে। তখন:

1. **Multiplexing (Sender site):**

- একাধিক process → একটাই UDP ব্যবহার করে ডাটা পাঠায়।
- UDP process-গুলোকে **port number** দিয়ে আলাদা করে চেনে।
- এরপর header যোগ করে ডাটাগ্রাম IP তে পাঠায়।

2. **Demultiplexing (Receiver site):**

- Receiver এ UDP ডাটাগ্রাম পায়।
- Header দেখে (বিশেষ করে port number), কোন process কে ডেলিভার করবে সেটা ঠিক করে।

👉 সহজ করে বললে — Port number হলো **address marker**, যা দিয়ে UDP বুঝতে পারে কোন ডাটাগ্রাম কোন app/process এর জন্য।

◇ UDP এর Features

1) Connectionless Service

- প্রতিটি ডাটাগ্রাম আলাদা, কোনটা আগে কোনটা পরে যাবে সেটা গ্যারান্টি নেই।
- সুবিধা:
 - ছোট ছোট request/response এর জন্য খুব দ্রুত কাজ করে।
 - যেমন DNS → query পাঠালাম, এক লাইন উত্তর পেলাম। কনেকশন না করেই কাজ শেষ।
- অসুবিধা:
 - বড় মেসেজ (যেমন SMTP দিয়ে ইমেইল পাঠানো) অনেক ডাটাগ্রামে ভাগ হয়ে গেলে সেগুলো অর্ডার মেনে পৌঁছাবে না।

2) No Error Control (Unreliable Service)

- UDP নিজে থেকে retransmission করে না।
- মানে কোন প্যাকেট lost/corrupted হলে সেটা বাদ দিয়েই বাকিগুলো deliver করে।

উদাহরণ:

- ☒ **Large file download (text, pdf)** → এখানে সব ডাটা দরকার, missing হলে চলবে না → তাই TCP ভালো।
- ☒ **Real-time video streaming / voice call** → কিছু প্যাকেট lost হলেও চলবে। Missing frame হলে একটু ঝাপসা বা এক সেকেন্ড ব্ল্যাংক, কিন্তু ভিডিও চালু থাকবে। তাই UDP ভালো।

3) No Congestion Control

- TCP তে congestion হলে retransmission হয়, আবারও নেটওয়ার্ক ব্যস্ত হয়।
 - UDP retransmission করে না, তাই congestion situation এ UDP অতিরিক্ত চাপ তৈরি করে না।
 - অর্থাৎ high-traffic নেটওয়ার্কে UDP অনেক সময় advantage.
-

◇ Typical Applications of UDP

UDP বেশিরভাগ সময় ব্যবহার হয় যেসব ক্ষেত্রে speed > reliability:

1. **DNS (Domain Name System)** → ছোট query/response → UDP perfect.
2. **SNMP (Simple Network Management Protocol)** → নেটওয়ার্ক ম্যানেজমেন্টে UDP ব্যবহার হয়।
3. **RIP (Routing Information Protocol)** → Routing update পাঠাতে UDP ব্যবহার হয়।
4. **TFTP (Trivial File Transfer Protocol)** → এর নিজের error control আছে, তাই UDP তে চলে।
5. **Multicasting** → UDP তে multicast support আছে (TCP তে নেই)।
6. **Real-time apps** (voice call, video call, live streaming) → TCP ব্যবহার করলে delay হয়ে যায়, তাই UDP ব্যবহার হয়।

◇ তুলনা (TCP vs UDP) সংক্ষেপে

বিষয়	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable (error control, retransmission)	Unreliable (no retransmission)
Delay	বেশি (কারণ connection setup + retransmission)	কম (দ্রুত response)
Use case	File transfer, email, web browsing	DNS, video call, live stream, VoIP

👉 সহজ করে মনে রাখো:

- **UDP = Fast but Unreliable** (যেখানে speed জরুরি, reliability তেমন না)।
- **TCP = Reliable but Slow** (যেখানে data 100% দরকার, delay সমস্যা না)।

◇ UDP Package এর ৫ টা Component

1. Control-Block Table

- এটা একটা টেবিল যেখানে UDP open ports এর তথ্য রাখে।
- প্রতিটি এন্ট্রিতে থাকে:
 - **State** → FREE (ব্যবহৃত না) / IN-USE (চলমান)

- **Process ID** → কোন process এই port ব্যবহার করছে
- **Port Number** → process এর জন্য assigned port
- **Queue Number** → ওই process এর সাথে সংযুক্ত queue (ডাটা রাখার জন্য)

👉 মানে, এটা UDP এর “directory” বা “address book” যেটা বলে দেয় কোন port কোন process এর সাথে যুক্ত।

2. Input Queues

- প্রতিটি process এর জন্য আলাদা queue থাকে (FIFO structure)।
 - যখন ডাটাগ্রাম আসে, UDP সেটাকে সঠিক queue তে রাখে।
 - এখানে output queue নেই (কারণ UDP সরাসরি process → IP পাঠায়, আলাদা করে queue বানায় না)।
-

3. Control-Block Module

- Control-block table কে manage করে।
 - যখন নতুন কোন process UDP ব্যবহার করতে চায়:
 1. Process → OS কে port number এর জন্য request করে।
 2. OS → port number assign করে (server এর জন্য well-known ports, client এর জন্য ephemeral ports)।
 3. Control-block module → table এ নতুন entry যোগ করে।
 4. শুরুতে queue number = 0 (মানে queue এখনও allocate করা হয়নি)।
-

4. Input Module

- IP থেকে UDP ডাটাগ্রাম রিসিভ করে।
- কাজের ধাপ (pseudo-code টেবিল 14.3 অনুযায়ী):
 1. ডাটাগ্রামের port number দেখে control-block table এ match খোঁজে।
 2. যদি match পাওয়া যায়:
 - দেখে queue allocate করা হয়েছে কিনা।
 - যদি না হয় → নতুন queue allocate করে।
 - এরপর ডাটা ওই queue তে enqueue করে।
 3. যদি match না পায়:
 - ICMP কে বলে একটা “unreachable port” error message পাঠাতে।
 - তারপর ওই datagram discard করে দেয়।

👉 মানে, input module হলো সেই system যেটা incoming UDP datagram কে ঠিক process এর queue তে পৌঁছে দেয়।

5. Output Module

- Sender side এ কাজ করে।
 - Process যখন UDP কে ডাটা দেয়:
 1. UDP header add করে (source port, destination port, length, checksum)।
 2. তারপর IP layer এ পাঠিয়ে দেয়।
 - Output side এ queue ব্যবহার করা হয়নি (কারণ UDP খুব lightweight রাখতে চায়)।
-

◇ Full Flow (Sending & Receiving এ UDP এর কাজ)

Sending (Output Module)

Process → UDP (header যোগ করে) → IP → Network

Receiving (Input Module)

Network → IP → UDP → control-block table দেখে → process এর input queue তে ডাটা deliver

◇ সহজ উদাহরণ

ধরো, তোমার কম্পিউটারে দুইটা process চলছে:

- Process A (DNS client, port 53 ব্যবহার করছে)
- Process B (Video streaming app, port 5000 ব্যবহার করছে)

এখন যদি DNS response আসে port 53 দিয়ে:

- Input Module → control-block table এ দেখে port 53 কে কোন process ব্যবহার করছে → Process A এর queue তে ডাটা পাঠায়।

আবার যদি video stream এর packet আসে port 5000 দিয়ে:

- সেটা Process B এর queue তে enqueue হবে।

কিন্তু যদি হঠাৎ করে একটা UDP packet আসে port 9999 দিয়ে (যেটা কোনো process ব্যবহার করছে না):

- তখন Input Module ICMP কে বলে “**unreachable port**” message পাঠাবে এবং packet discard করবে।

👉 এক কথায়:

Control-block table → কে কোন port ব্যবহার করছে জানায়

Input queues → process অনুযায়ী ডাটা জমা রাখে

Input module → packet কে সঠিক queue তে পাঠায়

Output module → packet তৈরি করে IP কে পাঠায়

Nice — I’ll put each **question** followed immediately by its **answer**.

1. In cases where reliability is not of primary importance, UDP would make a good transport protocol. Give examples of specific cases.

Answer:

UDP is a good choice when low latency, low overhead, or simple request/response semantics matter more than guaranteed delivery. Examples:

- **Real-time multimedia** (VoIP, live audio/video streaming) — occasional packet loss is preferable to delay.
 - **Interactive online games** — low latency is critical; missing a few updates is acceptable.
 - **DNS queries** — short request/response, want minimal delay and overhead.
 - **DHCP and other simple network control protocols** (small messages, retries handled at application level).
 - **TFTP (Trivial FTP)** — simple file transfer protocol that uses UDP and implements its own acknowledgements.
 - **Simple real-time telemetry** / sensor data streaming where stale data is worthless.
-

2. Are both UDP and IP unreliable to the same degree? Why or why not?

Answer:

Not exactly the *same degree*, but both are *connectionless and best-effort* (i.e., neither provides retransmission, ordering, or flow control).

Key differences:

- **IP (network layer)** provides best-effort packet delivery across networks: packets may be lost, duplicated, or delivered out of order. **IP does not provide end-to-end checksums** for the payload (IPv4 has header checksum only; IPv6 has none).
- **UDP (transport layer)** sits on top of IP and adds a small header and an optional end-to-end checksum (mandatory in IPv6, optional in IPv4 where a checksum value of 0 means “no checksum”). **UDP does not provide retransmission, ordering, or flow control — so it inherits IP’s unreliability for delivery.**

So: both are unreliable in that neither corrects losses or reorders — UDP provides slightly more end-to-end error detection and message boundary semantics, but not reliability mechanisms.

3. Show the entries for the header of a UDP user datagram that carries a message from a TFTP client to a TFTP server. Fill the checksum field with 0s. Choose an appropriate ephemeral port number and the correct well-known port number. The length of data is 40 bytes. Show the UDP packet using the format in Figure 14.2.

Question (restated): Show UDP header entries for a datagram from a TFTP client → TFTP server, checksum = 0, choose an ephemeral source port, TFTP well-known port as destination, data length 40 bytes; show packet in Figure 14.2 format.

Answer:

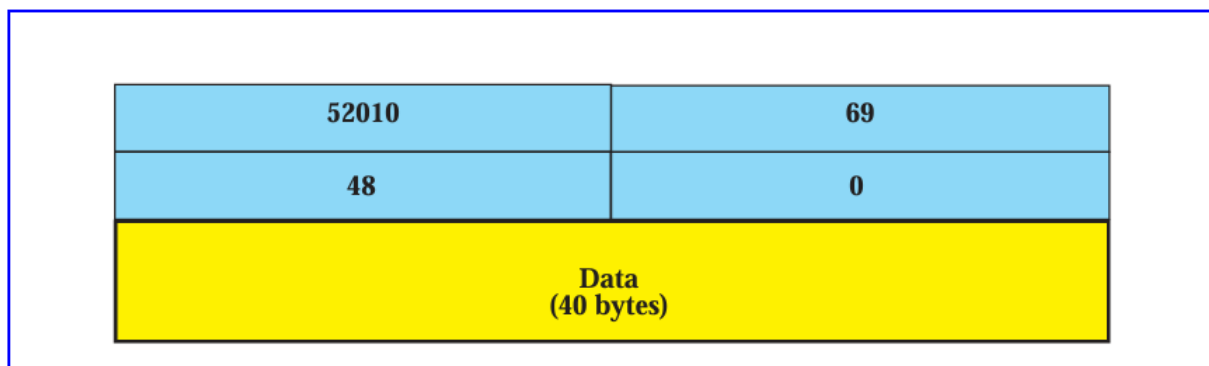
Choose an ephemeral source port: **52020** (one common ephemeral).

TFTP well-known UDP port: **69**.

UDP header is 8 bytes; data = 40 bytes → UDP **Length** = 8 + 40 = **48** bytes.

Presenting the UDP packet in the standard 4-field layout (each field 16 bits):

Figure 14.E3 *Solution to Exercise 3*



4. An SNMP client residing on a host with IP address 122.45.12.7 sends a message to an SNMP server residing on a host with IP address 200.112.45.90. What is the pair of sockets used in this communication?

Answer:

SNMP agents/servers use **UDP port 161 (well-known)**. The SNMP client will use an ephemeral source port. An example socket pair (client→server):

- **Client socket (source):** 122.45.12.7:49153 (example ephemeral port)
- **Server socket (destination):** 200.112.45.90:161

So the communication socket pair is:

(122.45.12.7:49153) → (200.112.45.90:161)

(Any ephemeral source port in the ephemeral range is acceptable; I used 49153 as an example.)

5. A TFTP server residing on a host with IP address 130.45.12.7 sends a message to a TFTP client residing on a host with IP address 14.90.90.33. What is the pair of sockets used in this communication?

Answer:

TFTP initial requests are to UDP port **69** on the server. Important TFTP note: when a client sends a request, it uses an ephemeral source port (e.g., `Cport`) and sends to server `s:69`. The server then responds **from a newly allocated ephemeral server port (not port 69) back to the client source port**; subsequent data/ACKs use that server ephemeral port.

Two possibilities, depending on whether this is the **initial request** or a **subsequent data/ACK** transfer:

- **Initial request (client → server):**
 - Client socket: 14.90.90.33:4970 (example ephemeral client port)
 - Server socket: 130.45.12.7:69
 - Pair: (14.90.90.33:4970) → (130.45.12.7:69)
- **Server response/data transfer (server uses a new ephemeral port):**
 - After receiving request, server chooses ephemeral port (example 130.45.12.7:6969) and sends back to client's 4970.
 - Pair for data packets: (130.45.12.7:6969) → (14.90.90.33:4970)

So if the question expects the simple socket pair for an outgoing server message (e.g., a reply) you can show the reply pair: (130.45.12.7:6969) → (14.90.90.33:4970). If it expects the initial contact pair, it's (14.90.90.33:4970) → (130.45.12.7:69).

(I used concrete ephemeral ports as examples — any valid ephemeral ports could be used in place of 49152/49153/4970/6969.)

6(a). What is the minimum size of a UDP datagram?

Answer:

Minimum UDP datagram size = 8 bytes (the UDP header only — 4 fields × 2 bytes each).

Explanation: UDP has a fixed 8-byte header; a datagram with zero payload is allowed, so 8 bytes is the minimum.

6(b). What is the maximum size of a UDP datagram?

Answer:

- **Protocol field limit (UDP length field): maximum UDP datagram length = 65,535 bytes** (this count **includes** the 8-byte UDP header).
- **Practical/when carried in IPv4:** because the IPv4 total-length field is also 16 bits, the largest IP datagram is 65,535 bytes. Subtracting a minimum IP header (20 bytes) gives a maximum UDP datagram inside IPv4 = **65,515 bytes**.

So you can say: protocol-level max = **65,535 bytes**; practical IPv4-encapsulated max = **65,515 bytes**.

6(c). What is the minimum size of the process data that can be encapsulated in a UDP datagram?

Answer:

Minimum process data size = 0 bytes (an empty payload is allowed). The UDP datagram would then be the 8-byte header only.

6(d). What is the maximum size of the process data that can be encapsulated in a UDP datagram?

Answer:

- **Protocol-level (UDP length field): max payload = $65,535 - 8 = 65,527$ bytes.**
- **Practical/IPv4 encapsulation:** IP total max $65,535 - \text{IP header } 20 - \text{UDP header } 8 = 65,507$ bytes.

Most texts quote **65,507 bytes** as the maximum UDP payload for IPv4 ($65,535 - 20 - 8$).

7. A client uses UDP to send data to a server. The data length is 16 bytes. Calculate the efficiency of this transmission at the UDP level (ratio of useful bytes to total bytes).

Question (restated): Useful bytes = application data = 16 bytes. At UDP level total = UDP header + data.

Answer (with arithmetic):

- UDP header = 8 bytes.
- Total UDP datagram = 8 + 16 = 24 bytes.
- Efficiency = useful / total = 16 / 24.

Digit-by-digit (fraction → percent):

$$16 / 24 = (16 \div 24) = (2 \div 3) = 0.666666... = 66.666...%$$

Rounded to two decimals: 66.67%.

8. Redo Exercise 7, calculating the efficiency of transmission at the IP level. Assume no options for the IP header.

Answer (with arithmetic):

- IP header (no options) = 20 bytes.
- IP packet includes IP header + UDP datagram (which is 24 bytes from above).
- Total at IP level = 20 + 24 = 44 bytes.
- Efficiency = useful / total = 16 / 44.

Digit-by-digit:

$$16 / 44 = (\text{divide numerator \& denominator by 4}) = 4 / 11 \approx 0.363636... = 36.3636...%$$

Rounded: 36.36%.

9. Redo Exercise 7, calculating the efficiency of transmission at the data link layer. Assume no options for the IP header and use Ethernet at the data link layer.

Assumptions & notes: I use the common Ethernet frame overhead = 14-byte Ethernet header + 4-byte FCS = 18 bytes of link-layer overhead. (Preamble/SFD/IFG typically not included in this calculation unless asked.)

Answer (with arithmetic):

- IP packet size (from Q8) = **44 bytes**.
- Ethernet overhead = 18 bytes.
- Total Ethernet frame = 44 + 18 = **62 bytes**.
- Efficiency = useful / total = 16 / 62.

Digit-by-digit:

$16 / 62 = \text{simplify dividing by 2} \rightarrow 8 / 31 \approx 0.2580645 = \mathbf{25.80645\%}$

Rounded: **25.81%**.

10. Given UDP header (in hexadecimal):

00 45 DF 00 00 58 FE 20

Each UDP header field is **16 bits (2 bytes)** long, and there are **4 fields total**:

Field	Bytes	Meaning
1	00 45	Source Port
2	DF 00	Destination Port
3	00 58	Length
4	FE 20	Checksum

(a) What is the source port number?

00 45 (in hex) =

$0 \times 256 + 69 = \mathbf{69 \text{ (decimal)}}$

☒ **Source port = 69**

(b) What is the destination port number?

DF 00 (in hex) =

$(0xDF \times 256) + 0x00 = (223 \times 256) + 0 = \mathbf{57088 \text{ (decimal)}}$

☒ **Destination port = 57088**

(c) What is the total length of the user datagram?

00 58 (in hex) = $(0 \times 256) + 0x58 = 88$ (decimal) bytes

☒ Total UDP length = 88 bytes

(d) What is the length of the data?

UDP header = 8 bytes

Data length = Total length – Header length
= $88 - 8 = 80$ bytes

☒ Data length = 80 bytes

(e) Is the packet directed from a client to a server or vice versa?

The **source port** = 69, which is a *well-known port number* assigned to **TFTP (Trivial File Transfer Protocol)**.

The **destination port** = 57088, which is an **ephemeral (temporary)** port, typically assigned to a client.

That means the **server (TFTP)** is sending a message **to the client**.

☒ **Direction:** From server → client

(f) What is the client process?

Since this is a TFTP message and the destination port (57088) belongs to the client, the client process is the **TFTP client**.

☒ **Client process = TFTP client**
